

GUJARAT TECHNOLOGICAL UNIVERSITY

Navyug Vidyabhavan Trust

C. K. Pithawalla College of Engineering and Technology, Surat

A

Project Report

On

Sentiment Analysis

Prepared as a part of the requirements for the subject of

Python for Data Science (BE05000231)

B. E. III, Semester – V

(Computer Engineering)

Submitted by:

Sr. No.	Enrolment No	Name of Student	Division
1	240090107021	Patil Darshna Satish	C
2	240090107094	Mistry Mahek Devndrabhai	C
3	240090107095	Mistry Vaidehi Dharmeshkumar	C
4	240090107098	More Heer Sanjay	C
5	240090107152	Patel Tisha Rohitkumar	C

Academic year (2026-27)

Table of Contents

Sr. No.	Title	Page No.
1	Problem Statement	2
2	Research/Idea development	3
3	Implementation (code/model/experiment)	9
	Conclusion and Future Work	41
	References	42

Problem Statement

Topic : Sentiment Analysis of Smartphone Reviews

1.1 Problem Statement

Online mobile-phone reviews contain useful information about customer experiences, but reading hundreds of reviews manually is time-consuming. The project addresses the problem of organizing and analysing review text together with product ratings and brands. The system classifies reviews into Positive, Negative, and Neutral sentiments and performs descriptive, statistical, visualization, natural-language-processing, database, and machine-learning tasks.

The project uses a dataset of 500 mobile-phone reviews containing brand, rating, review text, price, and sentiment-related information. The analysis is intended to identify sentiment distribution, compare brands, study the relationship between rating and sentiment, preprocess review text, convert text into numerical TF-IDF features, and train a Logistic Regression classifier for sentiment prediction.

Research/Idea development

2.1 Project Overview

Sentiment Analysis of Mobile Phone Reviews Using Python is a data science project that analyses customer reviews of mobile phones to understand their opinions and experiences. The project uses a dataset containing **500 mobile phone reviews** with information such as brand, rating, review text, price, and sentiment. Python libraries including **Pandas, NumPy, SciPy, Matplotlib, NLTK, and Scikit-learn** are used for data processing, statistical analysis, visualization, natural language processing, and machine learning.

The project begins with data loading, inspection, and preprocessing, including handling missing values, checking duplicates, cleaning review text, tokenization, stop-word removal, and lemmatization. Exploratory analysis is performed to study sentiment distribution, brand-wise reviews, ratings, and other patterns. Statistical techniques such as probability, sampling, descriptive statistics, and hypothesis testing are also applied. For machine learning, **TF-IDF** is used to convert review text into numerical features, followed by a **Logistic Regression** model for sentiment classification. The model achieved **88% accuracy** on the test dataset. The project demonstrates how Python can be used to extract meaningful insights from real-world customer review data.

2.2 Objective

The main objective of this project is to analyse customer reviews of mobile phones using Python and understand the opinions expressed by customers. The project focuses on classifying reviews into Positive, Negative, and Neutral sentiments and identifying patterns based on mobile-phone brands and customer ratings. It also aims to preprocess review text using Natural Language Processing techniques such as tokenization, stop-word removal, and lemmatization. Statistical analysis, probability, sampling, and hypothesis testing are performed to obtain meaningful insights from the dataset. Furthermore, the project uses TF-IDF to convert review text into numerical features and applies Logistic Regression to build a sentiment classification model. The project also demonstrates important Python concepts such as file handling, exception handling, custom modules, data visualization, and SQLite CRUD operations.

2.3 Scope

The scope of this project is to analyse customer reviews of mobile phones and extract useful information about customer opinions, ratings, and brand-wise performance. The project focuses on the available review dataset and uses Python-based techniques for data cleaning, exploratory data analysis, natural language processing, statistical analysis, data visualization, and machine learning. The analysis covers sentiment classification into Positive, Negative, and Neutral categories and examines the relationship between ratings, brands, and sentiments. The project also demonstrates supporting Python concepts such as file handling, exception handling, custom modules, and SQLite CRUD operations.

2.4 Key Outcomes

The project provides a structured analysis of the mobile phone review dataset and identifies patterns in customer sentiment, ratings, and brands. The analysis classifies the reviews into Positive, Negative, and Neutral categories and presents the results through statistical analysis and visualizations. The project also applies Natural Language Processing techniques to clean and prepare review text for further analysis. TF-IDF is used to convert the processed text into numerical features, which are then used to train a Logistic Regression model for sentiment classification. The trained model achieves an accuracy of **88%** on the test dataset. Overall, the project demonstrates the practical use of Python data science, NLP, statistical analysis, visualization, and machine learning techniques for analysing customer review data.

2.5 Dataset Used

The project uses a dataset containing **500 mobile phone customer reviews**. The dataset includes information related to the mobile phone, customer rating, review text, brand, price, cleaned review text, and sentiment. The review text is used as the main source for Natural Language Processing and sentiment analysis, while the rating and brand fields are used for statistical and comparative analysis.

The dataset contains reviews from different mobile phone brands, including **Apple, Samsung, OnePlus, Redmi, Oppo, Nokia, realme, Google Pixel, Vivo, and Motorola**. Before performing the analysis, the dataset is inspected for missing values and duplicate records. Missing price values are handled using the median, while records without review text or sentiment information are removed. After preprocessing, the dataset is used for exploratory analysis, visualization, statistical analysis, NLP processing, and machine-learning-based sentiment classification.

2.6 Python Core Concepts

The project uses several core Python concepts to perform data analysis and process customer reviews. Variables and expressions are used to store and manipulate data, while conditional statements are used for decision-making and validation. Loops are used to process multiple records and perform repetitive operations. Functions are created to perform specific and reusable tasks, such as cleaning review text and calculating sentiment percentages. Lists and dictionaries are used to store and organize data, while list and dictionary comprehensions are used for efficient data processing. String operations and regular expressions are also used extensively for cleaning and preparing the review text for Natural Language Processing.

2.7 Python Libraries

The project uses several Python libraries to perform different stages of data analysis and machine learning. **Pandas** is used for loading, cleaning, organizing, and analysing the dataset. **NumPy** is used for numerical calculations and statistical operations, while **SciPy** is used for statistical analysis and hypothesis testing. **Matplotlib** is used to create visualizations such as pie charts and bar charts for presenting the analysis results. **NLTK (Natural Language**

Toolkit) is used for Natural Language Processing tasks such as tokenization, stop-word removal, and lemmatization. **Scikit-learn** is used for TF-IDF feature extraction, train-test splitting, Logistic Regression, and model evaluation. The **SQLite3** library is used to demonstrate database operations such as creating, inserting, reading, updating, and deleting review records.

2.8 Custom Modules

A custom Python module named **sentiment_analysis.py** is used in the project to organize reusable functions related to sentiment analysis. The module contains functions for calculating percentages and determining the percentage of Positive, Negative, and Neutral reviews. The `calculate_percentage()` function calculates the percentage of a given count with respect to the total number of records, while the `sentiment_percentage()` function calculates the percentage of each sentiment category from the dataset. The custom module is imported into the main program and used to perform sentiment-related calculations, demonstrating the use of modular programming in Python.

2.9 Exception Handling

Exception handling is used in the project to handle errors that may occur during the execution of the program. The try and except blocks are used to prevent the program from terminating unexpectedly when an error occurs. In the project, exception handling is applied while loading the dataset, converting values into numeric form, accessing required columns, processing review text, and performing calculations. Different exceptions such as `FileNotFoundError`, `ValueError`, `KeyError`, and `ZeroDivisionError` are handled according to the operation being performed. This makes the program more reliable and allows meaningful error messages to be displayed when an unexpected situation occurs.

2.10 Web Scraping

Web scraping is a technique used to collect information automatically from websites. In the context of this project, web scraping could be used to collect mobile phone reviews, ratings, prices, and other product-related information from online sources. However, **web scraping is not implemented in the provided project notebook**, so it is not included as an implemented part of the project's results.

2.11 File Handling

File handling is used in the project to read and write data files. The dataset is loaded from an Excel file using the Pandas library, which allows the data to be accessed and processed as a `DataFrame`. After performing data cleaning and sentiment analysis, the processed results can be stored in a new Excel file for further use. This demonstrates how Python can be used to manage external data files during a data-science project.

2.12 CRUD Operations

The project demonstrates **CRUD (Create, Read, Update, and Delete) operations** using the SQLite database. A database table is created to store review information, and records are inserted into the table using the Create operation. The Read operation is used to retrieve and display stored review records. The Update operation modifies existing information in a review record, such as changing its rating. The Delete operation removes a selected record from the database. These operations demonstrate how Python can be used to manage structured review data using a relational database.

2.13 NumPy

NumPy is used in the project for performing numerical operations and statistical calculations on the dataset. It provides efficient array-based operations that are useful for analysing numerical values such as ratings and prices. In the project, NumPy is used to calculate values such as the mean, minimum, and maximum of numerical data. It helps simplify numerical analysis and supports the statistical processing performed during the project.

2.14 SciPy

SciPy is used in the project for performing statistical analysis on the dataset. It provides statistical functions that help analyse the distribution of numerical data and perform hypothesis testing. In the project, SciPy is used to obtain descriptive statistical information and to perform the **Chi-Square test of independence** between rating and sentiment. The statistical results help in understanding the relationship between the variables present in the dataset.

2.15 Pandas

Pandas is one of the main Python libraries used in this project for handling and analysing the mobile phone review dataset. It is used to load the Excel dataset into a DataFrame and inspect its structure, columns, data types, missing values, and duplicate records. Pandas is also used for data cleaning, including handling missing values and removing duplicate records. Various operations such as filtering records, grouping data by brand, calculating sentiment counts, and creating cross-tabulations between ratings and sentiments are performed using Pandas. This makes it easier to organize the dataset and prepare it for further statistical analysis, visualization, and machine learning.

2.16 Data Preprocessing

Data preprocessing is performed to prepare the mobile phone review dataset for analysis and machine learning. The dataset is first checked for duplicate records and missing values. Missing values in the `actual_price` column are handled using the median value, while records without required `review_text` or sentiment information are removed. The review text is then converted into lowercase and unwanted characters and punctuation are removed. Natural Language Processing techniques are applied to tokenize the text, remove stop words, and perform

lemmatization. The resulting cleaned review text is stored for further analysis and is later used for TF-IDF feature extraction and sentiment classification.

2.17 Statistical Analysis

Statistical analysis is performed to understand the numerical characteristics and patterns present in the mobile phone review dataset. Descriptive statistics are used to analyse values such as ratings and prices, including the minimum, maximum, mean, and variance. The project also performs brand-wise analysis to compare customer ratings among different mobile phone brands. For example, the Samsung reviews contain 64 records, with ratings ranging from 1 to 5 and an average rating of approximately **3.05**. These statistical measures help in understanding the distribution and overall characteristics of the dataset.

2.18 Probability

Probability analysis is used to determine the likelihood of each sentiment category occurring in the dataset. The project contains a total of **500 reviews**, including **209 Positive, 191 Negative, and 100 Neutral** reviews. Based on these values, the probability of a randomly selected review being Positive is **0.418**, Negative is **0.382**, and Neutral is **0.200**. This analysis provides a numerical representation of the sentiment distribution in the dataset and helps in understanding the relative occurrence of each sentiment category.

2.19 Sampling & Inference

Sampling is used in the project to select a smaller group of records from the complete dataset for analysis. From the total **500 reviews**, a random sample of **50 reviews** is selected using a fixed random state. The sample contains **28 Positive, 12 Negative, and 10 Neutral** reviews. The corresponding sample proportions are **0.56 Positive, 0.24 Negative, and 0.20 Neutral**. These sample results are compared with the proportions of the complete dataset to demonstrate how the distribution of a sample can differ from the overall dataset.

2.20 Hypothesis Testing

Hypothesis testing is used in the project to determine whether there is a significant relationship between **rating and sentiment**. A **Chi-Square Test of Independence** is applied to the rating-by-sentiment contingency table. The test produces a Chi-Square statistic of approximately **1000** and a very small p-value of approximately 1.49×10^{-210} . Using a significance level of **0.05**, the null hypothesis is rejected in the notebook, indicating a statistically significant association between rating and sentiment in the analysed dataset.

2.21 Machine Learning

The project uses machine learning to classify mobile phone reviews according to their sentiment. The review text is first converted into numerical features using **TF-IDF (Term Frequency–Inverse Document Frequency)**. The dataset is divided into training and testing sets using an **80:20 split**, with 400 records used for training and 100 records used for testing.

A **Logistic Regression** model is then trained using the TF-IDF features to predict whether a review is Positive, Negative, or Neutral.

The trained model achieves an **accuracy of 88%** on the test dataset. Precision, recall, and F1-score are also calculated for each sentiment category to evaluate the classification performance. The confusion matrix is used to identify correctly and incorrectly classified reviews. The machine-learning implementation demonstrates how text data can be transformed into numerical features and used to build a sentiment classification model.

Implementation (code/model/experiment)

The following section presents the implementation of the project with selected code and corresponding outputs. The screenshots are arranged according to the workflow: dataset loading, preprocessing, analysis, NLP, visualization, database operations, and machine learning. Each figure is accompanied by a short explanation of its purpose.

3.1 Dataset Loading



```
import pandas as pd

try:
    data = pd.read_excel("/content/Dataset.xlsx")
    print("Dataset loaded successfully!")

except FileNotFoundError:
    print("Error: Dataset file not found.")

except Exception as e:
    print("An error occurred:", e)
```

... Dataset loaded successfully!

The Excel dataset is loaded using Pandas. Exception handling is included to handle a missing dataset file.

3.2 DataFrame Creation

Implementation workflow: Dataset Loading → Data Cleaning → NLP Preprocessing → Statistical Analysis → Visualization → TF-IDF → Logistic Regression → Evaluation → SQLite CRUD.

```
#create a dataframe
df = pd.DataFrame(data)
print("Original DataFrame:")
print(df.head())
```

The loaded data is converted into a Pandas DataFrame, providing a structured format for further processing and analysis.

3.3: Initial Dataset Preview

The first records of the dataset are displayed to verify the columns and values before analysis

```
*** Original DataFrame:
   No.  actual_price  rating \
0    1         7700.0        2
1    2        30700.0        3
2    3         21000.0        1
3    4         23200.0        2
4    5         23200.0        5

   review_text  brand \
0  This rating is not for the product.... Its for...  Apple
1  The product is usable and matches the basic de...  Nokia
2                                dont buy  realme
3                                Phone is superb  Samsung
4  My data cable started to getting damage!!!  Samsung

   cleaned_review_text  sentiment
0  rating product seller delivered product scratc...  Negative
1  hanging issue network dropping issue needed re...  Neutral
2                                dont buy  Negative
3                                phone superb  Negative
4  data cable started getting damage  Positive
```

3.4: File Handling Using Pandas

The project demonstrates reading the Excel dataset and writing processed data to an Excel file.

```
from pandas.io.api import read_excel
#file Handling

#read an excel file
data = pd.read_excel("/content/Dataset.xlsx")
data.head()

#write a file
data.to_excel(
    "/content/Sentiment_Results.xlsx",
    index=False
)

print("Data saved successfully!")

*** Data saved successfully!
```

3.5: Dataset Information

The DataFrame information is inspected to verify the number of records, columns, non-null values, and data types.

```
data.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 7 columns):
 #   Column                Non-Null Count  Dtype  
---  -
 0   No.                   500 non-null   int64  
 1   actual_price          498 non-null   float64
 2   rating               500 non-null   int64  
 3   review_text          500 non-null   object  
 4   brand                500 non-null   object  
 5   cleaned_review_text  497 non-null   object  
 6   sentiment            500 non-null   object  
dtypes: float64(1), int64(2), object(4)
memory usage: 27.5+ KB
```

3.6: Dataset Head

The head() function displays the first five records for an initial inspection of the dataset.

```
data.head()
```

	No.	actual_price	rating	review_text	brand	cleaned_review_text	sentiment
0	1	7700.0	2	This rating is not for the product.... Its for...	Apple	rating product seller delivered product scratc...	Negative
1	2	30700.0	3	The product is usable and matches the basic de...	Nokia	hanging issue network dropping issue needed re...	Neutral
2	3	21000.0	1	dont buy	realme	dont buy	Negative
3	4	23200.0	2	Phone is superb	Samsung	phone superb	Negative
4	5	23200.0	5	My data cable started to getting damage!!!	Samsung	data cable started getting damage	Positive

3.7: Brand and Sentiment Count

Value counts are used to examine the distribution of mobile-phone brands and sentiment categories

```
print(data["brand"].value_counts())
print(data["sentiment"].value_counts())
```

```
brand
Apple      104
Samsung     64
OnePlus     51
Redmi       49
Oppo        44
Nokia       42
realme      38
Google Pixel 37
Vivo        36
Motorola    35
Name: count, dtype: int64
sentiment
Positive    209
Negative    191
Neutral     100
Name: count, dtype: int64
```

3.8: Missing Values and Duplicate Check

The dataset is checked for missing values and duplicate records before further processing.

```
▶ #check the missing values
print(data.isnull().sum())

# Remove duplicate records
data = data.drop_duplicates()

#check for any duplicate values
print("Duplicates:", data.duplicated().sum())

# Remove rows with missing review text
data = data.dropna(subset=["review_text", "sentiment"])

print("Rows:", data.shape[0])
print("Columns:", data.shape[1])
```

```

... No.                0
   actual_price        2
   rating             0
   review_text        0
   brand              0
   cleaned_review_text 3
   sentiment          0
   dtype: int64
   Duplicates: 0
   Rows: 500
   Columns: 7

```

3.9: Handling Missing Values

Missing values in the price field are handled using the median, and the resulting dataset dimensions are checked.

```

#fill the null values

data["actual_price"] = data["actual_price"].fillna(
    data["actual_price"].median()
)

print(data.isnull().sum())

print("Rows:", data.shape[0])
print("Columns:", data.shape[1])

```

```

No.                0
actual_price        0
rating             0
review_text        0
brand              0
cleaned_review_text 3
sentiment          0
dtype: int64
Rows: 500
Columns: 7

```

3.10: Exception Handling for Rating Values

The rating column is converted to numeric form and ValueError is handled if an invalid value is encountered.

```
try:
    df["rating"] = pd.to_numeric(
        df["rating"],
        errors="raise"
    )

    print("Rating data is valid.")

except ValueError:
    print("Error: Invalid rating value found.")

except Exception as e:
    print("Unexpected error:", e)

... Rating data is valid.
```

3.11: Exception Handling for Sentiment Column

KeyError handling is used to detect whether the required sentiment column exists.

```
try:
    sentiments = df["sentiment"]
    print("Sentiment column found.")

except KeyError:
    print("Error: Sentiment column does not exist.")

... Sentiment column found.
```

3.12: Sentiment Filtering and Counting

Sentiment categories are counted and positive reviews are filtered for further analysis.

```
1  ▶ #Count reviews by sentiment
   sentiment_count = df["sentiment"].value_counts()

   print("Sentiment Count")
   print(sentiment_count)

   #Filter positive reviews
   positive_reviews = df[df["sentiment"] == "Positive"]

   #Filter negative reviews
   negative_reviews = df[df["sentiment"] == "Negative"]

   #Filter neutral reviews
   neutral_reviews = df[df["sentiment"] == "Neutral"]

   #Analyze sentiment according to rating
   sentiment_rating = pd.crosstab(
       df["rating"],
       df["sentiment"]
   )
   print("")
   print(sentiment_rating)

   #Find average rating for each sentiment
   avg_rating_sentiment = df.groupby("sentiment")["rating"].mean()

   print("")
   print(avg_rating_sentiment)
```

```
*** Sentiment Count
sentiment
Positive    209
Negative    191
Neutral     100
Name: count, dtype: int64

sentiment  Negative  Neutral  Positive
rating
1           151         0         0
2            40         0         0
3             0        100         0
4             0         0         39
5             0         0        170

sentiment
Negative    1.209424
Neutral     3.000000
Positive    4.813397
Name: rating, dtype: float64
```

3.13: Samsung Review Analysis

Samsung reviews are filtered to analyse their ratings and sentiment-related information.

```
▶ samsung_reviews = df[df["brand"] == "Samsung"]

samsung_positive = df[
    (df["brand"] == "Samsung") &
    (df["rating"] >= 4)
]

samsung = df[df["brand"] == "Samsung"]

print("Total Samsung reviews:", len(samsung))
print("Average Samsung rating:", samsung["rating"].mean())
print("Average Samsung price:", samsung["actual_price"].mean())
samsung_sentiment = samsung.groupby("sentiment").size()

print(samsung_sentiment)
print(df["brand"].unique())
print(df["brand"].value_counts())

*** Total Samsung reviews: 64
Average Samsung rating: 3.046875
Average Samsung price: 35179.25
sentiment
Negative    25
Neutral     13
Positive    26
dtype: int64
['Apple' 'Nokia' 'realme' 'Samsung' 'OnePlus' 'Oppo' 'Redmi' 'Motorola'
 'Vivo' 'Google Pixel']
brand
Apple          104
Samsung         64
OnePlus         51
Redmi           49
Oppo            44
Nokia           42
realme          38
Google Pixel    37
Vivo            36
Motorola        35
Name: count, dtype: int64
```

3.14: Apple Sentiment Analysis

Apple reviews are selected and their sentiment distribution is calculated using Pandas

```
apple_df = df[df["brand"] == "Apple"]

# Sentiment distribution for Apple
apple_sentiment = apple_df["sentiment"].value_counts()

print(apple_sentiment)

apple_analysis = pd.crosstab(
    apple_df["rating"],
    apple_df["sentiment"]
)

print(apple_analysis)

avg_rating = apple_df.groupby("sentiment")["rating"].mean()

print(avg_rating)

apple_df["review_length"] = apple_df["review_text"].astype(str).str.len()

review_length = apple_df.groupby("sentiment")["review_length"].mean()

print(review_length)
```

```
*** sentiment
Negative    48
Positive    46
Neutral     10
Name: count, dtype: int64
sentiment  Negative  Neutral  Positive
rating
1           37         0         0
2           11         0         0
3            0        10         0
4            0         0         7
5            0         0        39

sentiment
Negative    1.229167
Neutral     3.000000
Positive    4.847826
Name: rating, dtype: float64
sentiment
Negative    240.708333
Neutral     56.800000
Positive    75.326087
Name: review_length, dtype: float64
/tmp/ipykernel_3057/802301114.py:19: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

3.15: Brand-wise Average Rating

Average ratings are calculated for each brand and sorted to compare brand-wise rating performance.

```
▶ #Which brands have the highest average rating?
brand_analysis = df.groupby("brand")["rating"].mean().sort_values(ascending=False)

print("")
print(brand_analysis)

#Which brands have the most reviews?
brand_reviews = df["brand"].value_counts()
print("")
print(brand_reviews)
```

```
...
brand
Oppo          3.522727
Motorola      3.200000
Vivo          3.194444
realme        3.184211
Redmi         3.061224
Samsung       3.046875
OnePlus       3.000000
Apple         3.000000
Nokia         2.880952
Google Pixel  2.783784
Name: rating, dtype: float64

brand
Apple         104
Samsung       64
OnePlus       51
Redmi         49
Oppo          44
Nokia         42
realme        38
Google Pixel  37
Vivo          36
Motorola      35
Name: count, dtype: int64
```

3.16: Sentiment Probability Calculation

The probability of Positive, Negative, and Neutral reviews is calculated from their frequencies in the complete dataset.

```
try:
    total = len(df)

    positive = len(df[df["sentiment"] == "Positive"])
    negative = len(df[df["sentiment"] == "Negative"])
    neutral = len(df[df["sentiment"] == "Neutral"])

    print("Positive probability:", positive / total)
    print("Negative probability:", negative / total)
    print("Neutral probability:", neutral / total)

except ZeroDivisionError:
    print("Error: Dataset contains no records.")
```

```
*** Positive probability: 0.418
    Negative probability: 0.382
    Neutral probability: 0.2
```

3.17: Brand-wise Sentiment Probability

Sentiment proportions are calculated for a selected mobile-phone brand.

```
brand_df = df[df["brand"] == "Apple"]

total = len(brand_df)

positive = len(brand_df[brand_df["sentiment"] == "Positive"])
negative = len(brand_df[brand_df["sentiment"] == "Negative"])
neutral = len(brand_df[brand_df["sentiment"] == "Neutral"])

print("Total Apple reviews:", total)

print("P(Positive):", positive / total)
print("P(Negative):", negative / total)
print("P(Neutral):", neutral / total)
```

```
*** Total Apple reviews: 104
    P(Positive): 0.4423076923076923
    P(Negative): 0.46153846153846156
    P(Neutral): 0.09615384615384616
```

3.18: Random Sampling

A random sample of 50 reviews is selected using a fixed random state, and its sentiment distribution is analysed.

```
sample_df = df.sample(n=50, random_state=42)
print("Sample size:", len(sample_df))

sample_sentiment = sample_df["sentiment"].value_counts()
print(sample_sentiment)

sample_probability = sample_df["sentiment"].value_counts(normalize=True)
print(sample_probability)

full_probability = df["sentiment"].value_counts(normalize=True)

print("Full dataset:")
print(full_probability)

print("\nSample:")
print(sample_probability)
```

```
Sample size: 50
sentiment
Positive    28
Negative    12
Neutral     10
Name: count, dtype: int64
sentiment
Positive    0.56
Negative    0.24
Neutral     0.20
Name: proportion, dtype: float64
Full dataset:
sentiment
Positive    0.418
Negative    0.382
Neutral     0.200
Name: proportion, dtype: float64

Sample:
sentiment
Positive    0.56
Negative    0.24
Neutral     0.20
Name: proportion, dtype: float64
```

3.19: Chi-Square Test of Independence

A rating-by-sentiment contingency table is created and the Chi-Square test is applied to examine their association.

```
from scipy.stats import chi2_contingency

table = pd.crosstab(df["rating"], df["sentiment"])

print(table)

chi2, p_value, dof, expected = chi2_contingency(table)

print("Chi-Square Statistic:", chi2)
print("P-value:", p_value)

if p_value < 0.05:
    print("Reject the null hypothesis.")
    print("There is a significant relationship between rating and sentiment.")
else:
    print("Fail to reject the null hypothesis.")
    print("There is no significant relationship between rating and sentiment.")
```



```
... sentiment  Negative  Neutral  Positive
rating
1           151         0         0
2           40         0         0
3            0        100         0
4            0         0         39
5            0         0        170
Chi-Square Statistic: 999.9999999999999
P-value: 1.4932281660407321e-210
Reject the null hypothesis.
There is a significant relationship between rating and sentiment.
```

3.20: Custom Python Module

The custom `sentiment_analysis.py` module contains reusable functions for calculating sentiment percentages.

```
#custom module
%%writefile sentiment_analysis.py

def calculate_percentage(count, total):
    """Calculate percentage."""

    if total == 0:
        return 0

    return (count / total) * 100

def sentiment_percentage(data):
    """Calculate percentage of each sentiment."""

    total = len(data)

    positive = len(data[data["sentiment"] == "Positive"])
    negative = len(data[data["sentiment"] == "Negative"])
    neutral = len(data[data["sentiment"] == "Neutral"])

    return {
        "Positive": round(calculate_percentage(positive, total), 2),
        "Negative": round(calculate_percentage(negative, total), 2),
        "Neutral": round(calculate_percentage(neutral, total), 2)
    }
```

Overwriting `sentiment_analysis.py`

3.21: NumPy Numerical Analysis

NumPy arrays are used to perform numerical calculations on Samsung ratings and prices.

```
▶ import numpy as np

# Select Samsung products
samsung = data[data["brand"] == "Samsung"]

# Convert columns into NumPy arrays
ratings = np.array(samsung["rating"].dropna())
prices = np.array(samsung["actual_price"].dropna())

# Calculate statistics
avg_rating = np.mean(ratings)
highest_price = np.max(prices)
lowest_price = np.min(prices)

print("Samsung Analysis")
print("Average Rating:", avg_rating)
print("Highest Rating:", np.max(ratings))
print("Lowest Rating:", np.min(ratings))
print("\nHighest Price:", highest_price)
print("Lowest Price:", lowest_price)
```

```
... Samsung Analysis
Average Rating: 3.046875
Highest Rating: 5
Lowest Rating: 1

Highest Price: 125600.0
Lowest Price: 6995.0
```

3.22: Sentiment Percentage Analysis

Sentiment counts are converted into percentages to compare the distribution of sentiment categories.

```
print(samsung["sentiment"].value_counts())
percentage = (
    samsung["sentiment"].value_counts(normalize=True) * 100
)

print(percentage.round(2))
```

```
sentiment
Positive    26
Negative    25
Neutral     13
Name: count, dtype: int64
sentiment
Positive    40.62
Negative    39.06
Neutral     20.31
Name: proportion, dtype: float64
```

3.23: SciPy Descriptive Statistics

SciPy statistical functions are used to calculate descriptive statistics for Samsung ratings.

```
from scipy import stats

# Select Samsung ratings
samsung_ratings = data[
    data["brand"] == "Samsung"
][["rating"]].dropna()

# Descriptive statistics
result = stats.describe(samsung_ratings)

print("Number of observations:", result.nobs)
print("Minimum:", result.minmax[0])
print("Maximum:", result.minmax[1])
print("Mean:", result.mean)
print("Variance:", result.variance)

samsung_ratings = data[
    data["brand"] == "Samsung"
][["rating"]].dropna()
```

```
... Number of observations: 64
Minimum: 1
Maximum: 5
Mean: 3.046875
Variance: 2.616815476190476
```

3.24: NLTK Setup for NLP

NLTK resources are downloaded and prepared for tokenization, stop-word removal, and lemmatization.

```
from nltk.sentiment import SentimentIntensityAnalyzer

sia = SentimentIntensityAnalyzer()

review = "Amazing phone with excellent camera, fast performance, and great battery life!"

score = sia.polarity_scores(review)

print("Review:", review)
print("Negative:", score["neg"])
print("Neutral:", score["neu"])
print("Positive:", score["pos"])
print("Compound:", score["compound"])

# Final sentiment
if score["compound"] >= 0.05:
    sentiment = "Positive"
elif score["compound"] <= -0.05:
    sentiment = "Negative"
else:
    sentiment = "Neutral"

print("Final Sentiment:", sentiment)
```

```
Review: Amazing phone with excellent camera, fast performance, and great battery life!
Negative: 0.0
Neutral: 0.402
Positive: 0.598
Compound: 0.9168
Final Sentiment: Positive
```

3.25: Text Cleaning and Preprocessing

The review text is cleaned and the original and cleaned versions are displayed for comparison.



```
import nltk
```

```
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')
nltk.download('wordnet')
```

```
... [nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Package wordnet is already up-to-date!
True
```

```
data["cleaned_review_text"] = data["review_text"].apply(clean_text)
print(data[["review_text", "cleaned_review_text"]].head(5))
```

```
review_text \
0 This rating is not for the product.... Its for...
1 The product is usable and matches the basic de...
2 dont buy
3 Phone is superb
4 My data cable started to getting damage!!!
```

```
cleaned_review_text
0 rating product seller delivered product scratc...
1 product usable match basic description
2 dont buy
3 phone superb
4 data cable started getting damage
```

3.26: Cleaned Review Text

The `cleaned_review_text` field is generated from the original review text for subsequent NLP and machine-learning processing.

```
data["cleaned_review_text"] = data["review_text"].apply(clean_text)
print(data[["review_text", "cleaned_review_text"]].head(5))
```

	review_text	cleaned_review_text
0	This rating is not for the product.... Its for...	rating product seller delivered product scratc...
1	The product is usable and matches the basic de...	product usable match basic description
2	dont buy	dont buy
3	Phone is superb	phone superb
4	My data cable started to getting damage!!!	data cable started getting damage

3.27 Sentiment Analysis Example

A sample review is processed with NLTK sentiment-analysis functionality to demonstrate sentiment scoring.

```
from nltk.sentiment import SentimentIntensityAnalyzer

sia = SentimentIntensityAnalyzer()

review = "Amazing phone with excellent camera, fast performance, and great battery life!"

score = sia.polarity_scores(review)

print("Review:", review)
print("Negative:", score["neg"])
print("Neutral:", score["neu"])
print("Positive:", score["pos"])
print("Compound:", score["compound"])

# Final sentiment
if score["compound"] >= 0.05:
    sentiment = "Positive"
elif score["compound"] <= -0.05:
    sentiment = "Negative"
else:
    sentiment = "Neutral"

print("Final Sentiment:", sentiment)
```

```
... Review: Amazing phone with excellent camera, fast performance, and great battery life!
Negative: 0.0
Neutral: 0.402
Positive: 0.598
Compound: 0.9168
Final Sentiment: Positive
```

3.28: TF-IDF Vectorization

TfidfVectorizer converts the cleaned review text into numerical TF-IDF features for machine learning.

```
# TF-IDF Vectorization

from sklearn.feature_extraction.text import TfidfVectorizer

# Create TF-IDF Vectorizer
vectorizer = TfidfVectorizer(
    max_features=5000,
    ngram_range=(1, 2)
)

# Convert cleaned text into numerical vectors
X = vectorizer.fit_transform(data["cleaned_review_text"])

print("TF-IDF Vectorization completed successfully!")
print("Shape of vectorized data:", X.shape)
```

```
... TF-IDF Vectorization completed successfully!
    Shape of vectorized data: (500, 3848)
```

3.29: TF-IDF Feature Names

The generated TF-IDF feature names are inspected to understand the vocabulary used by the model.

```
# Get feature names

features = vectorizer.get_feature_names_out()

print("Number of features:", len(features))

print("\nFirst 20 features:")
print(features[:20])
```

```
• Number of features: 3848

First 20 features:
['aapple' 'aapple bad' 'ability' 'ability due' 'able' 'able activate'
 'able recieve' 'able return' 'able unlock' 'able use' 'abruptly'
 'abruptly annoying' 'absolute' 'absolute joy' 'absolutely'
 'absolutely perfect' 'acceptable' 'acceptable room' 'accepting'
 'accepting bad']
```

3.30: TF-IDF Feature Matrix

The TF-IDF matrix is displayed as a DataFrame so that textual reviews can be represented numerically.

```
# Display TF-IDF vectors
```

```
tfidf_df = pd.DataFrame(  
    X.toarray(),  
    columns=features  
)
```

```
print(tfidf_df.head())
```

```
***
```

	aaple	aaple	bad	ability	ability	due	able	able	activate	able	recieve	\
0	0.0		0.0	0.0		0.0	0.0		0.0		0.0	
1	0.0		0.0	0.0		0.0	0.0		0.0		0.0	
2	0.0		0.0	0.0		0.0	0.0		0.0		0.0	
3	0.0		0.0	0.0		0.0	0.0		0.0		0.0	
4	0.0		0.0	0.0		0.0	0.0		0.0		0.0	

	able	return	able	unlock	able	use	...	yesterday	yesterday	place	\
0		0.0		0.0		0.0	...		0.0		0.0
1		0.0		0.0		0.0	...		0.0		0.0
2		0.0		0.0		0.0	...		0.0		0.0
3		0.0		0.0		0.0	...		0.0		0.0
4		0.0		0.0		0.0	...		0.0		0.0

	yesterday	voice	yet	yet	start	youre	youre	vested	zero	zero	noise	\
0		0.0	0.0		0.0	0.0		0.0	0.0		0.0	
1		0.0	0.0		0.0	0.0		0.0	0.0		0.0	
2		0.0	0.0		0.0	0.0		0.0	0.0		0.0	
3		0.0	0.0		0.0	0.0		0.0	0.0		0.0	
4		0.0	0.0		0.0	0.0		0.0	0.0		0.0	

	zero	quality
0		0.0
1		0.0
2		0.0
3		0.0
4		0.0

```
[5 rows x 3848 columns]
```

3.32: Positive Reviews by Brand

Positive reviews are grouped by mobile-phone brand to compare the number of positive customer opinions.

```
import matplotlib.pyplot as plt

# Select only positive reviews
positive_reviews = data[data["sentiment"] == "Positive"]

# Count positive reviews for each brand
positive_by_brand = positive_reviews["brand"].value_counts()
top_3_brands = positive_by_brand.head(3)

total_positive = len(positive_reviews)

print("Top 3 Brands with Highest Positive Reviews:")

for brand, count in top_3_brands.items():
    percentage = (count / total_positive) * 100
    print(
        brand,
        ":", count,
        "reviews (" ,
        round(percentage, 2),
        "%)"
    )

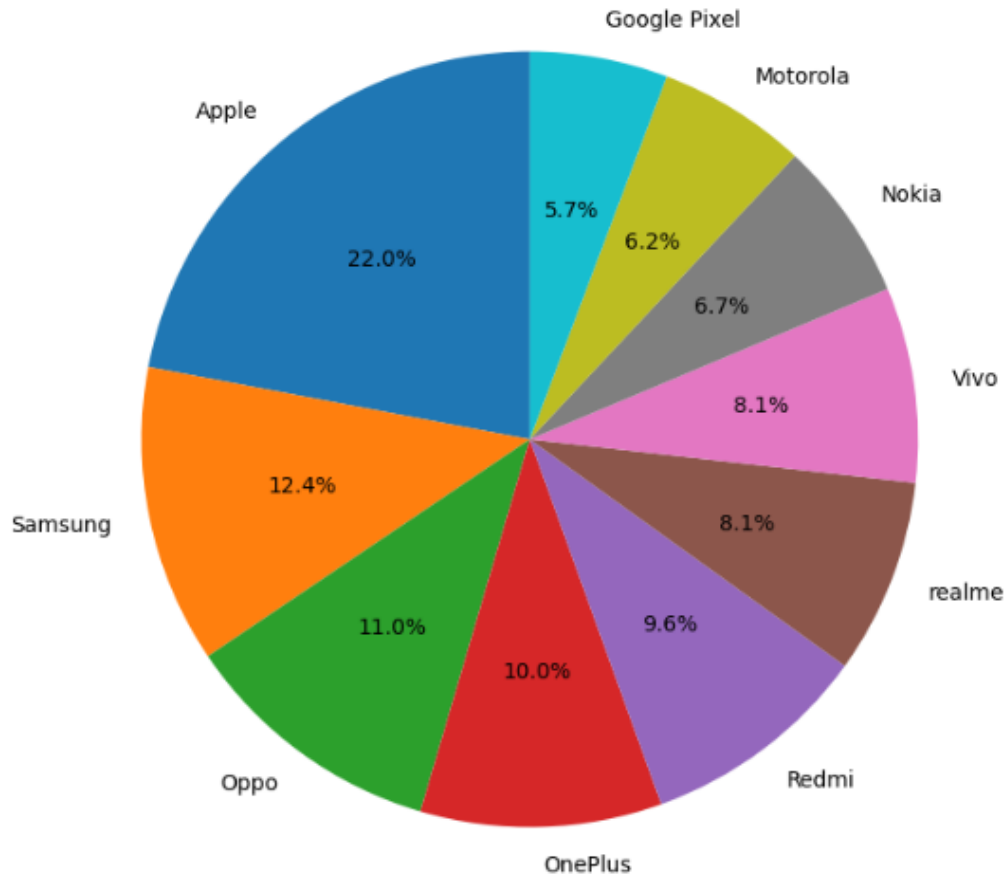
# Pie chart
plt.figure(figsize=(8, 8))

plt.pie(
    positive_by_brand.values,
    labels=positive_by_brand.index,
    autopct="%1.1f%%",
    startangle=90
)

plt.title("Positive Reviews by Phone Brand")
plt.show()
```

*** Top 3 Brands with Highest Positive Reviews:
Apple : 46 reviews (22.01 %)
Samsung : 26 reviews (12.44 %)
Oppo : 23 reviews (11.0 %)

Positive Reviews by Phone Brand



3.33: Samsung Sentiment Distribution

A pie chart presents the distribution of sentiment categories among Samsung reviews.

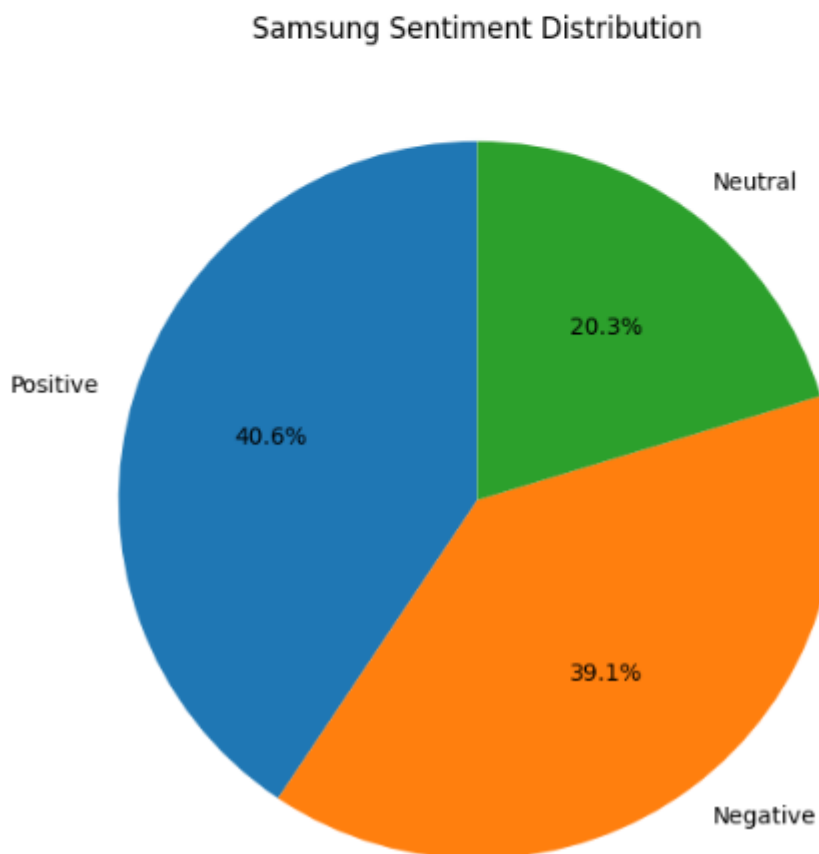
```
# Select Samsung reviews
samsung = data[data["brand"] == "Samsung"]

# Count Samsung sentiments
samsung_sentiment = samsung["sentiment"].value_counts()

# Pie chart
plt.figure(figsize=(7, 7))

plt.pie(
    samsung_sentiment.values,
    labels=samsung_sentiment.index,
    autopct="%1.1f%%",
    startangle=90
)

plt.title("Samsung Sentiment Distribution")
plt.show()
```



3.34: Top Brands with Positive Reviews

The top brands by number of positive reviews are identified and visualized.

```
▶ import matplotlib.pyplot as plt

# Select only positive reviews
positive_reviews = data[data["sentiment"] == "Positive"]

# Count positive reviews for each brand
positive_by_brand = positive_reviews["brand"].value_counts()
top_3_brands = positive_by_brand.head(3)

total_positive = len(positive_reviews)

print("Top 3 Brands with Highest Positive Reviews:")

for brand, count in top_3_brands.items():
    percentage = (count / total_positive) * 100
    print(
        brand,
        ":", count,
        "reviews (",
        round(percentage, 2),
        "%)"
    )

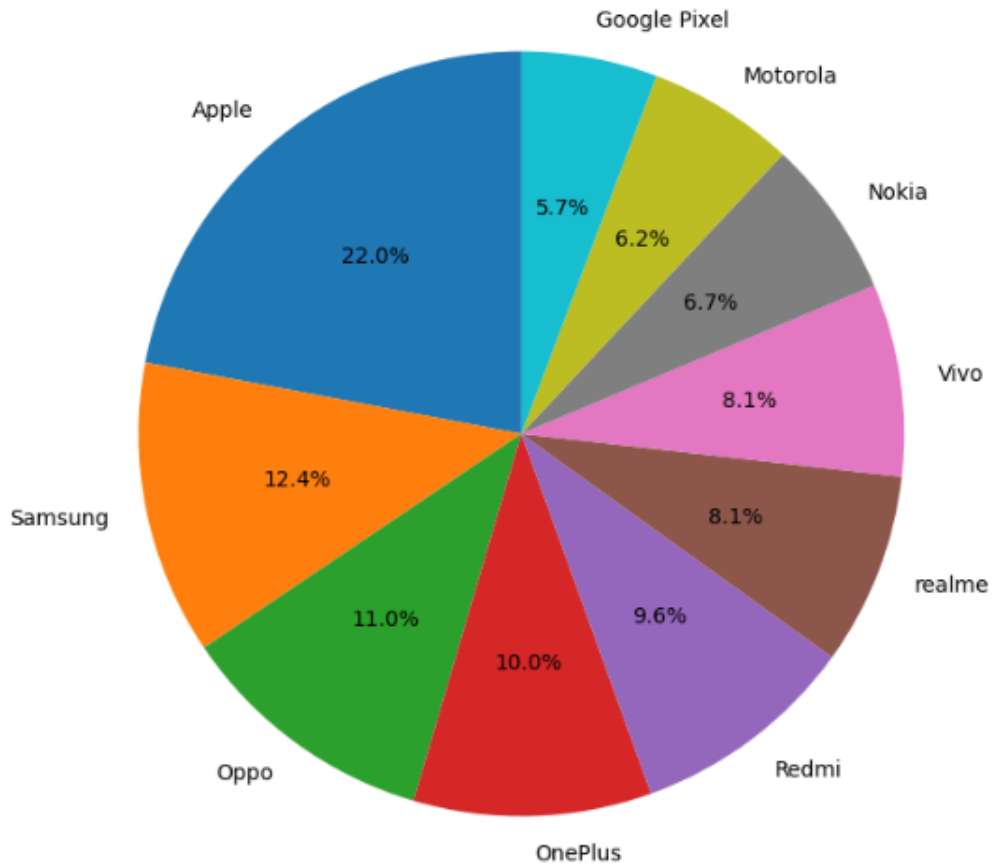
# Pie chart
plt.figure(figsize=(8, 8))

plt.pie(
    positive_by_brand.values,
    labels=positive_by_brand.index,
    autopct="%1.1f%%",
    startangle=90
)

plt.title("Positive Reviews by Phone Brand")
plt.show()
```

Top 3 Brands with Highest Positive Reviews:
Apple : 46 reviews (22.01 %)
Samsung : 26 reviews (12.44 %)
Oppo : 23 reviews (11.0 %)

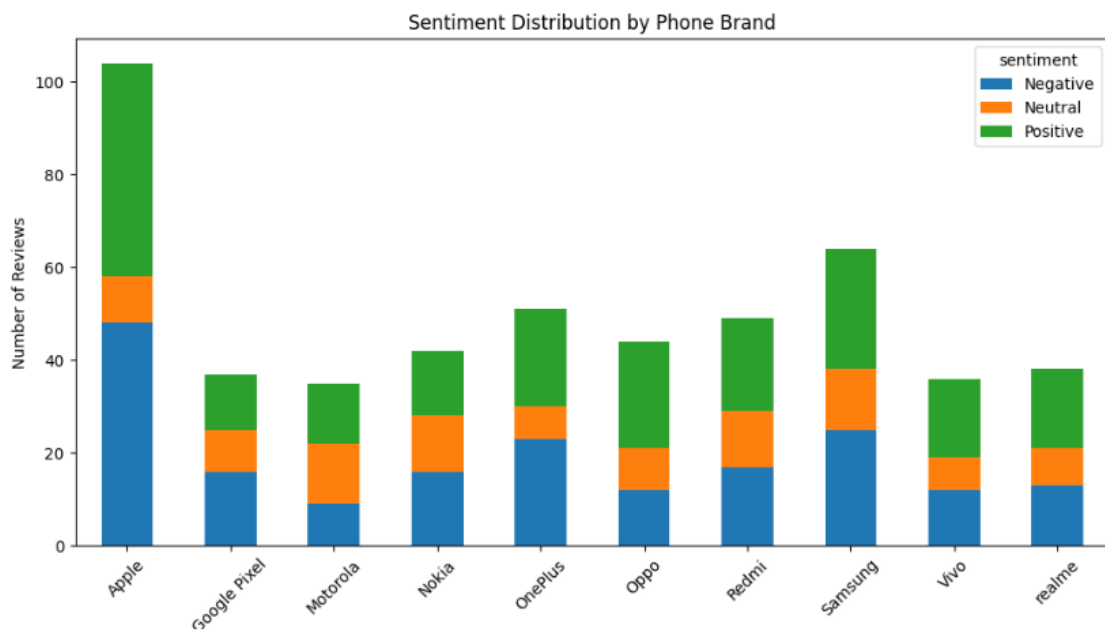
Positive Reviews by Phone Brand



3.35: Sentiment Distribution by Brand

A stacked bar chart compares Positive, Negative, and Neutral reviews across mobile-phone brands.

```
sentiment_by_brand = pd.crosstab(  
    data["brand"],  
    data["sentiment"]  
)  
  
sentiment_by_brand.plot(  
    kind="bar",  
    stacked=True,  
    figsize=(10, 6)  
)  
  
plt.title("Sentiment Distribution by Phone Brand")  
plt.xlabel("Phone Brand")  
plt.ylabel("Number of Reviews")  
plt.xticks(rotation=45)  
plt.tight_layout()  
plt.show()
```



3.36: Average Rating by Brand

Average customer ratings are calculated and plotted for each mobile-phone brand.

```
import matplotlib.pyplot as plt

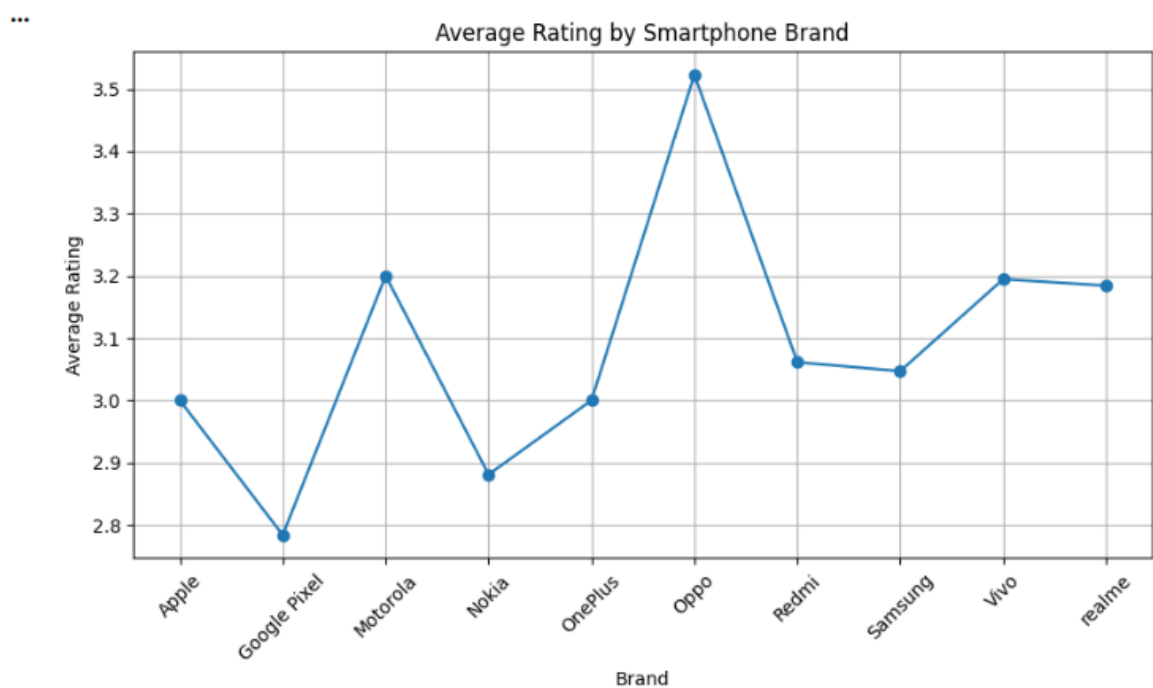
average_rating = df.groupby("brand")["rating"].mean()

plt.figure(figsize=(10, 5))

plt.plot(
    average_rating.index,
    average_rating.values,
    marker="o"
)

plt.title("Average Rating by Smartphone Brand")
plt.xlabel("Brand")
plt.ylabel("Average Rating")

plt.xticks(rotation=45)
plt.grid(True)
plt.show()
```



3.37: SQLite Database Connection

A SQLite database connection is created and a cursor is initialized for database operations.

```
import sqlite3

conn = sqlite3.connect("sentiment_analysis.db")
cursor = conn.cursor()
```

3.38: SQLite Table Creation

The reviews table is created in SQLite to store structured review information.

```
print("Database created successfully!")

cursor.execute("""
CREATE TABLE IF NOT EXISTS reviews (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    brand TEXT,
    rating INTEGER,
    review_text TEXT,
    sentiment TEXT
)
""")

conn.commit()

print("Reviews table created successfully!")
```

```
Database created successfully!
Reviews table created successfully!
```

3.39: SQLite Create/Insert Operation

Review records are inserted into the SQLite database using the Create operation.

```
five_reviews = data.head(5)
for _, row in five_reviews.iterrows():

    cursor.execute("""
INSERT INTO reviews
(brand, rating, review_text, sentiment)
VALUES (?, ?, ?, ?)
""", (
    row["brand"],
    row["rating"],
    row["review_text"],
    row["sentiment"]
))

conn.commit()

print("5 reviews added successfully!")
```

```
5 reviews added successfully!
```

v

3.40: SQLite Read Operation

Stored review records are retrieved from the database using a SELECT query.

```
cursor.execute("SELECT * FROM reviews")

for row in cursor.fetchall():
    print(row)

(2, 'Nokia', 4, 'The product is usable and matches the basic description.', 'Neutral')
(3, 'realme', 1, 'dont buy', 'Negative')
(4, 'Samsung', 2, ' Phone is superb ', 'Negative')
(5, 'Samsung', 5, 'My data cable started to getting damage!!!', 'Positive')
(6, 'Apple', 2, 'This rating is not for the product.... Its for the seller... Who has delivered the p', 'Negative')
(7, 'Nokia', 3, 'The product is usable and matches the basic description.', 'Neutral')
(8, 'realme', 1, 'dont buy', 'Negative')
(9, 'Samsung', 2, ' Phone is superb ', 'Negative')
(10, 'Samsung', 5, 'My data cable started to getting damage!!!', 'Positive')
```

3.41: Model Training and Prediction and Machine-Learning Results

The final results show the training and testing sizes, model accuracy, and classification report for sentiment prediction.

The Logistic Regression model is trained on the TF-IDF training features and used to predict sentiment for the test set.

```
▶ # MACHINE LEARNING - SENTIMENT ANALYSIS

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Load dataset
df = pd.read_excel("Dataset.xlsx")

# Remove rows where review text or sentiment is missing
df = df.dropna(subset=["review_text", "sentiment"])

# Input (X) and Target (y)
X = df["review_text"]
y = df["sentiment"]

# Split dataset into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
```

```

# Convert text into numerical features using TF-IDF
vectorizer = TfidfVectorizer()

X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)

# Create Machine Learning model
model = LogisticRegression(max_iter=1000)

# Train the model
model.fit(X_train_tfidf, y_train)

# Predict sentiment for test data
y_pred = model.predict(X_test_tfidf)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("==== MACHINE LEARNING RESULTS ====")
print("Training records:", len(X_train))
print("Testing records:", len(X_test))
print("Accuracy:", accuracy)

# Classification report
print("\n==== CLASSIFICATION REPORT ====")
print(classification_report(y_test, y_pred))

# Confusion matrix
print("\n==== CONFUSION MATRIX ====")
print(confusion_matrix(y_test, y_pred))

# Test the model with a new review
new_review = ["The phone has excellent camera quality and battery life"]

new_review_tfidf = vectorizer.transform(new_review)

prediction = model.predict(new_review_tfidf)

print("\n==== NEW REVIEW PREDICTION ====")
print("Review:", new_review[0])
print("Predicted Sentiment:", prediction[0])

```

===== MACHINE LEARNING RESULTS =====

Training records: 400

Testing records: 100

Accuracy: 0.88

===== CLASSIFICATION REPORT =====

	precision	recall	f1-score	support
Negative	0.84	0.84	0.84	38
Neutral	1.00	1.00	1.00	20
Positive	0.86	0.86	0.86	42
accuracy			0.88	100
macro avg	0.90	0.90	0.90	100
weighted avg	0.88	0.88	0.88	100

===== CONFUSION MATRIX =====

```
[[32  0  6]
 [ 0 20  0]
 [ 6  0 36]]
```

===== NEW REVIEW PREDICTION =====

Review: The phone has excellent camera quality and battery life

Predicted Sentiment: Positive

<https://github.com/vaidehimistry05/Sentiment-Analysis-Smartphone-Reviews>

Conclusion and Future Work

Conclusion

The project successfully demonstrates the use of Python for analysing mobile phone customer reviews and extracting meaningful information from the dataset. The data was cleaned and preprocessed before performing exploratory, statistical, and Natural Language Processing operations. Different Python libraries were used for data manipulation, numerical analysis, visualization, text processing, and machine learning. The sentiment analysis classified the reviews into Positive, Negative, and Neutral categories, while the statistical analysis helped identify patterns between ratings and sentiments. TF-IDF was used to convert review text into numerical features, and a Logistic Regression model was trained for sentiment classification. The model achieved an accuracy of 88% on the test dataset. Overall, the project demonstrates a complete data-science workflow from data preprocessing to analysis and machine learning.

Future Work

In the future, the project can be extended by using a larger and more diverse collection of customer reviews. More detailed analysis can be performed to identify specific aspects of mobile phones that customers appreciate or dislike, such as camera quality, battery life, performance, design, and price. The classification process can also be improved to handle more complex customer opinions and mixed sentiments. Additional visual analysis can be introduced to study trends over time, compare products in greater detail, and provide deeper insights into customer preferences.

References

Web Links

Python Documentation — <https://docs.python.org/>

Pandas Documentation — <https://pandas.pydata.org/docs/>

NumPy Documentation — <https://numpy.org/doc/>

SciPy Documentation — <https://docs.scipy.org/>

NLTK Documentation — <https://www.nltk.org/>

Scikit-learn Documentation — <https://scikit-learn.org/>

Matplotlib Documentation — <https://matplotlib.org/>

SQLite Documentation — <https://www.sqlite.org/docs.html>

Textbooks

Python programming and Data Science reference material.